

helixqa_integration.agent.outline

HelixQA Full Integration into HelixCode System — Master Plan

Executive Summary

Purpose and Scope

Full integration of HelixQA submodule (with all dependency submodules) into HelixCode enterprise CLI agent system, with Catalogizer as reference implementation

Anti-bluff guarantee: every test and Challenge must verify real end-user usability, not just pass green

100% coverage target across all supported test types and all client applications (Web, Desktop, Mobile, CLI, TUI)

On-demand screenshot capability for all client apps during HelixQA sessions

Key Findings from Research

HelixCode has 6 client applications (CLI/Cobra, TUI/tview, Desktop/Fyne, Android/gomobile, iOS/gomobile, Aurora/Harmony OS) across 5 platform categories

HelixQA has 40+ Go packages with autonomous QA, evidence collection, and LLM-powered bug detection — but gaps exist in iOS, Windows, macOS desktop, CLI/TUI visual capture

Catalogizer already includes HelixQA as submodule at commit 35deb43 (Phase 27.7), but is ~2 phases behind upstream (0bca023, Phase 29)

Article XI §11.9 user-mandate forensic anchor is MISSING from HelixCode governance files — critical compliance gap

CONST-035 anti-bluff testing rules are present in HelixQA and Catalogizer but cascade to submodules is unverified

1. Phase 0: Constitution & Governance Update (~2500 words, 2 tables)

1.1 Article XI §11.9 Cascade to HelixCode

1.1.1 GAP-001: Add verbatim user-mandate forensic anchor to HelixCode/CONSTITUTION.md at Article XI §11.9 — exact text to insert with user quote about tests passing but features broken

1.1.2 GAP-002: Add Article XI §11.9 operative rule to HelixCode/CLAUDE.md — “the bar for shipping is not ‘tests pass’ but ‘users can use the feature’”

1.1.3 GAP-003: Add Article XI §11.9 reference to HelixCode/AGENTS.md — binding autonomous agents to anti-bluff mandate

1.2 CONST-035 Naming Alignment

1.2.1 HelixCode uses CONST-017 for anti-bluff rules; rename to CONST-035 for cross-repository consistency

1.2.2 Update all internal references from CONST-017 to CONST-035 in HelixCode docs and code comments

1.3 Submodule Governance Cascade

1.3.1 Verify all 15 HelixCode submodules have CONSTITUTION.md, CLAUDE.md, AGENTS.md with CONST-035 and Article XI §11.9

1.3.2 Verify all 41 Catalogizer submodules have governance files with anti-bluff mandates

1.3.3 Create governance verification script: scripts/verify-governance-cascade.sh — checks every submodule for required files and mandated text

1.3.4 Add governance check to run_all_challenges.sh — block merge if any submodule lacks anti-bluff mandate

1.4 Exact File Changes Reference

1.4.1 Table: every file to modify, line range, exact insertion text, and verification method

2. Phase 1: Submodule Dependency Resolution (~3000 words, 3 tables)

2.1 HelixQA Dependency Map

2.1.1 HelixQA direct dependencies: digital.vasic.challenges (../Challenges), digital.vasic.containers (../Containers)

2.1.2 HelixQA autonomous session dependencies: LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor

2.1.3 HelixQA tools/opensource/ submodules: 25+ vendored OSS packages with license audit requirements

2.2 Submodule Registration in HelixCode

**2.2.1 Add HelixQA to HelixCode/.gitmodules with SSH URL:
git@github.com:HelixDevelopment/HelixQA.git**

2.2.2 Add HelixQA dependency submodules to .gitmodules: Challenges, Containers, LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor

2.2.3 Configure submodule paths: HelixQA/ at root, Dependencies/ for external tools

2.2.4 Version locking strategy: pin commits, bump via make bump-submodules with verification

2.3 Catalogizer Submodule Synchronization

2.3.1 Bump HelixQA from 35deb43 to latest 0bca023 — exact git commands and verification steps

2.3.2 Cascade bump to all HelixDevelopment submodules: DocProcessor 5f1e58a, LLMOrchestrator 1b95823, LLMProvider 0720b6e

2.3.3 Verify Challenges submodule compatibility: Catalogizer uses vasic-digital/Challenges at 4390e48; HelixQA needs digital.vasic.challenges — verify API compatibility

2.4 Build System Integration

2.4.1 Update HelixCode/Makefile: add helixqa-build, helixqa-test, helixqa-challenge targets

2.4.2 Update HelixCode/go.mod: add replace directive for digital.vasic.helixqa → ./HelixQA

2.4.3 Update Catalogizer/catalog-api/go.mod: verify replace directives cover all HelixQA dependency modules

2.4.4 Docker integration: update docker-compose.yml to include helixqa-runner service

3. Phase 2: HelixQA Core Integration into HelixCode (~4000 words, 4 tables, 1 diagram)

3.1 Integration Architecture

3.1.1 Design pattern: HelixQA as embedded library within HelixCode server, NOT as external service

3.1.2 Integration point: internal/helixqa/ package in HelixCode — wrapper around HelixQA orchestrator

3.1.3 API exposure: REST endpoints under /api/v1/qa/* for triggering QA sessions, retrieving reports, on-demand screenshots

3.1.4 CLI integration: extend Cobra command tree with `helixcode qa run`,
`helixcode qa report`, `helixcode qa screenshot`

3.2 Configuration Injection

3.2.1 Extend HelixCode/config/ to include QA configuration section: banks path, platforms, device IDs, coverage target

3.2.2 Merge HelixQA .env.example into HelixCode .env.example — unified environment configuration

3.2.3 Add QA config validation: verify bank files exist, devices are reachable, LLM API keys are set

3.3 Server-Side QA Endpoint Implementation

3.3.1 Add POST /api/v1/qa/session — start new QA session with platform selection and bank specification

3.3.2 Add GET /api/v1/qa/session/:id/status — real-time session progress with PhaseManager integration

3.3.3 Add GET /api/v1/qa/session/:id/report — retrieve completed QA report in markdown/html/json

3.3.4 Add GET /api/v1/qa/session/:id/screenshot/:name — on-demand screenshot retrieval with base64 or file path

3.3.5 Add DELETE /api/v1/qa/session/:id — cancel running session with cleanup

3.4 CLI Command Registration

3.4.1 Register qa subcommand in cmd/cli/main.go — create cmd/cli/qa.go with run, report, screenshot, list subcommands

3.4.2 Implement `helixcode qa run --banks --platforms --output`

3.4.3 Implement helixcode qa screenshot --session --platform --output

3.4.4 Implement helixcode qa report --session --format <markdown|html|json>

3.5 TUI Integration

3.5.1 Add QA session dashboard to terminal-ui/ — real-time progress display with tview table and status bars

3.5.2 Add screenshot preview pane in TUI — ASCII-art thumbnail or external viewer invocation

3.5.3 Key bindings: F5 start session, F6 view report, F7 view screenshots

4. Phase 3: On-Demand Screenshot Pipeline (~3500 words, 3 tables, 1 diagram)

4.1 Screenshot Architecture Design

4.1.1 New package: pkg/screenshot/ — Manager, Engine interface, per-platform Engine implementations

4.1.2 Engine interface methods: Capture(ctx, opts) ([]byte, error), Supports(platform) bool, Name() string

4.1.3 Storage interface: Save(name, data) (path, error), Load(name) ([]byte, error), List(sessionID) []string

4.2 Per-Platform Screenshot Implementation

4.2.1 Web: Playwright full-page + responsive breakpoints (mobile 375px, tablet 768px, desktop 1440px); chromedp alternative for headless

4.2.2 Desktop Linux: X11 via import -window root + xwd; Wayland via grim; multi-monitor via xrandr enumeration

4.2.3 Desktop macOS: screencapture -x (immediate) + ScreenCaptureKit (CGO, Phase 5.5)

4.2.4 Desktop Windows: DXGI Desktop Duplication API or BitBlt fallback (new implementation required)

4.2.5 Android: adb exec-out screencap -p with rotation detection; sccpy for high-speed capture

4.2.6 iOS: xcrun simctl io screenshot for simulator; go-ios / WebDriverAgent for real devices (new implementation)

4.2.7 CLI: asciinema recording + ANSI-to-PNG renderer; tmux capture-pane for TUI state reconstruction

4.2.8 TUI: xterm.js terminal state capture + html-to-image rendering; or direct terminal buffer dump

4.3 On-Demand Screenshot API

4.3.1 HTTP endpoint: GET /api/v1/qa/screenshot?platform=X&session=Y&format=base64|file|url

4.3.2 CLI command: helixcode qa screenshot --platform --session --responsive --dark-mode

4.3.3 Real-time streaming: WebSocket /ws/qa/screenshot for live session capture during autonomous QA

4.3.4 Presentational export: PowerPoint/Keynote-ready screenshot packs with annotations and timestamps

4.4 Anti-Bluff Visual Verification

4.4.1 Template matching: verify screenshot contains expected UI elements via OpenCV template matching or SSIM

4.4.2 OCR validation: extract text from screenshot via Tesseract, verify expected content is visible

4.4.3 Vision LLM validation: send screenshot to vision model, ask “does this show a working login screen?”

4.4.4 Deliberate-break test: break the feature, screenshot MUST show broken state (not cached/green)

5. Phase 4: Test Type & Challenge Coverage Matrix (~3500 words, 4 tables)

5.1 Test Type Definitions

5.1.1 Unit tests: mocks allowed only in *_test.go with go test -short; verify individual functions

5.1.2 Integration tests: real databases, real HTTP calls, real service interactions; no mocks

5.1.3 E2E tests: full user workflows across client → API → database → external services

5.1.4 Functional tests: feature-level validation with real user actions and visible outcomes

5.1.5 Security tests: penetration testing, CSRF, XSS, SQL injection, authentication bypass attempts

5.1.6 Stress tests: high concurrency, large payloads, memory pressure, connection exhaustion

5.1.7 Chaos tests: random failures, network partitions, service kills, database restarts mid-session

5.1.8 Benchmark tests: performance baselines, regression detection, latency percentiles

5.1.9 Challenge tests: real-life scenario scripts in challenges/scripts/ — validate actual behavior not return codes

5.1.10 Runtime verification: live system probes, health checks, circuit breaker validation

5.2 Client App Coverage Matrix

5.2.1 Table: 10 test types × 5 client categories (Web, Desktop, Mobile, CLI, TUI) = 50 cells with specific test approaches

5.2.2 Web client: Playwright-based E2E, visual regression, responsive breakpoint validation, accessibility (axe-core)

5.2.3 Desktop client: X11/Playwright automation, window state verification, multi-monitor capture, crash detection

5.2.4 Mobile client (Android): ADB-based UI automation, ANR/crash detection, geo-restriction probing, deep link validation

5.2.5 Mobile client (iOS): simctl / WebDriverAgent automation, screenshot diff, app lifecycle testing

5.2.6 CLI client: Command invocation with real output parsing, exit code verification, stdout/stderr content analysis, ANSI rendering capture

5.2.7 TUI client: Terminal state capture, keyboard interaction simulation, screen buffer comparison, color/attribute verification

5.3 Challenge Design Standards

5.3.1 Every Challenge must have: setup, execution, verification, teardown phases

5.3.2 Verification must use protocol-layer probes: TCP connect + real request + real response parsing (not just grep)

5.3.3 Visual Challenges: capture screenshot before/after action, use SSIM > 0.95 or vision LLM to verify state change

5.3.4 Deliberate-break validation: temporarily break the feature, Challenge MUST fail; restore, Challenge MUST pass

5.4 Coverage Measurement

5.4.1 Code coverage: go test -coverprofile with 100% target for critical paths

5.4.2 Feature coverage: map every user-facing feature to at least one Challenge and one E2E test

5.4.3 Platform coverage: every client app tested on every supported OS (Linux, macOS, Windows, Aurora, SymphonyOS)

5.4.4 Provider coverage: every LLM provider tested with at least one real inference call per release

6. Phase 5: Catalogizer Example Integration (~3000 words, 3 tables)

6.1 Current State Assessment

6.1.1 Catalogizer has HelixQA at 35deb43 (Phase 27.7) — 2 phases behind upstream 0bca023 (Phase 29)

6.1.2 Existing QA artifacts: 60+ test banks, 5 QA session archives, 206 PASS / 1 SKIP last audit

6.1.3 Integration gaps: Android/Android TV autonomous QA blocked; installer-wizard and API client lack HelixQA banks; OCU-CUDA-Sidecar not deployed

6.2 Bump and Synchronization Steps

6.2.1 Step 1: cd Catalogizer && git submodule update --remote HelixQA

6.2.2 Step 2: verify HelixQA builds: cd HelixQA && make build && make test

6.2.3 Step 3: bump all HelixDevelopment submodules: DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine

6.2.4 Step 4: run Catalogizer Full-QA Master Cycle: make qa-all with all 60+ banks

6.2.5 Step 5: verify no regressions: compare results to baseline 206 PASS / 1 SKIP

6.3 New Test Bank Creation

6.3.1 Create banks/catalogizer-web-functional.yaml: cover login, media browsing, search, favorites, settings

6.3.2 Create banks/catalogizer-desktop-e2e.yaml: cover Tauri app launch, window management, protocol connections

6.3.3 Create banks/catalogizer-api-contract.yaml: cover all REST endpoints with real HTTP requests and response validation

6.3.4 Create banks/catalogizer-android-tv.yaml: cover Android TV channel browsing, playback, deep links

6.3.5 Create banks/catalogizer-translation-workflow.yaml: cover subtitle management, language settings, Cyrillic/Unicode handling

6.4 Client App Validation

6.4.1 Web (Collection-Manager-React): Playwright automation with responsive breakpoints, dark mode, real-time WebSocket updates

6.4.2 Desktop (Tauri): window capture, menu interaction, protocol connection dialogs, offline/online transitions

6.4.3 Android (Kotlin): ADB-based navigation, media playback, search with Cyrillic terms, favorites sync

6.4.4 API (Go): contract testing with real HTTP calls, JWT auth flow, WebSocket subscription lifecycle

7. Phase 6: Anti-Bluff Testing Framework (~3500 words, 3 tables)

7.1 Anti-Bluff Architecture

7.1.1 Layer 1 — Protocol probes: TCP connect + real request + real response (not just TCP-open)

7.1.2 Layer 2 — Functional verification: execute real user action, verify real outcome (not just absence of error)

7.1.3 Layer 3 — Visual verification: screenshot before/after, SSIM comparison, OCR text extraction, vision LLM analysis

7.1.4 Layer 4 — Destructive validation: deliberately break the feature, confirm test fails; restore, confirm test passes

7.2 Anti-Bluff Test Implementation

7.2.1 Add pkg/antibluff/ to HelixQA: Validator interface, Probe implementations, Breaker harness

7.2.2 Implement ServiceProbe: start real service, send real request, parse real response, validate content

7.2.3 Implement VisualProbe: capture screenshot, run template matching / OCR / vision LLM, return confidence score

7.2.4 Implement BreakerHarness: intercept service calls, inject failures (timeout, 500, wrong response), verify detection

7.3 Constitution Compliance Verification

7.3.1 Add make anti-bluff target: runs all tests with deliberate-break validation

7.3.2 Add challenges/scripts/anti_bluff_challenge.sh: comprehensive anti-bluff validation suite

7.3.3 Add to run_all_challenges.sh: anti-bluff phase after all other challenges pass

7.3.4 Verification criteria: if any test passes while feature is deliberately broken, the test is non-conformant and blocks release

7.4 Synthetic User Workflows

7.4.1 Workflow 1 — Onboarding: new user registration → login → first media scan → dashboard view (all clients)

7.4.2 Workflow 2 — Media Management: browse → search “Титаник” → add to favorites → export favorites → verify file

7.4.3 Workflow 3 — Translation: open settings → change language → verify UI language change → search in new language

7.4.4 Workflow 4 — Protocol Resilience: disconnect SMB → verify offline indicator → reconnect → verify auto-sync

7.4.5 Each workflow: must have visual proof (screenshots), must fail if any step is broken, must cover all 5 client types

8. Phase 7: AI-Driven QA Session Orchestration (~3000 words, 2 tables, 1 diagram)

8.1 Autonomous Session Architecture

8.1.1 SessionCoordinator orchestrates 4-phase lifecycle: Setup → Doc-Driven Verification → Curiosity-Driven Exploration → Report & Cleanup

8.1.2 PlatformWorker per platform: acquires agent from AgentPool, runs analyzer, navigator, issueDetector

8.1.3 LLM selection: LLMsVerifier scores providers, picks strongest vision model across available hosts

8.1.4 Feature map building: DocProcessor reads project docs, builds coverage map, tracks verified vs unverified features

8.2 Heavy QA Session Design

8.2.1 Session trigger: manual (make qa-session), scheduled (cron-like via orchestrator), or event-driven (post-deploy hook)

8.2.2 Platform parallelism: run Web + Desktop + Mobile + CLI + TUI workers concurrently with isolated evidence dirs

8.2.3 Timeout and resource limits: 30-40% host resources per session (GOMAXPROCS=2, nice -n 19), 2h default timeout

8.2.4 Graceful degradation: if LLM provider unavailable, curiosity phase skips (not fakes); if device unavailable, platform worker skips

8.3 Report Generation and Distribution

8.3.1 QA report: markdown + HTML + JSON with platform results, crash/ANR counts, coverage percentage, ticket list

8.3.2 Video evidence: per-platform video with timeline event overlays, linked to tickets and screenshots

8.3.3 Ticket generation: markdown tickets with evidence paths, severity, reproduction steps, LLM-suggested fixes

8.3.4 Dashboard integration: challenges-dashboard static HTML over qa-results/ with pass/fail trends

8.4 Presentational Screenshots on Demand

8.4.1 During autonomous session: SessionRecorder captures screenshots at every step, indexes by platform/timestamp/step-name

8.4.2 On-demand API: GET /api/v1/qa/screenshot?session=X&platform=Y&step=Z returns specific screenshot

8.4.3 Presentational export: compile screenshots into annotated gallery with captions, timestamps, and pass/fail status

8.4.4 Slide deck generation: automated PowerPoint/Keynote export from QA session evidence for stakeholder presentations

9. Phase 8: Enterprise UX Validation (~2500 words, 2 tables)

9.1 Translation Tool UX Matrix

9.1.1 Web UX: language selector dropdown, real-time translation preview, provider selection, model quality indicator

9.1.2 Desktop UX: Tauri window with native OS integration, menu shortcuts, tray icon status, offline mode

9.1.3 Mobile UX: touch-optimized interface, voice input, camera OCR, push notifications for long translations

9.1.4 CLI UX: batch translation mode, progress bars, provider fallback chains, output formatting (json/csv/txt)

9.1.5 TUI UX: interactive provider selection, real-time streaming display, keyboard shortcuts, color-coded quality scores

9.2 Provider & Model Coverage

9.2.1 Tier 1 providers (native): Anthropic, OpenAI, Google Gemini, OpenRouter, DeepSeek, Groq, Ollama, Astica, Kimi, Qwen, Stepfun, NVIDIA, xAI, Mistral

9.2.2 Tier 2 providers (OpenAI-compatible): 30+ providers via unified adapter

9.2.3 Vision-capable providers: 12+ with score-based ranking and adaptive routing

9.2.4 QA must verify: every provider returns real translation, every model handles Cyrillic/Unicode, fallback chains work under failure

9.3 Enterprise Quality Requirements

9.3.1 Multi-tenancy: isolated sessions, per-tenant provider quotas, usage tracking

9.3.2 Reliability: 99.9% uptime target, circuit breakers for all providers, automatic failover

9.3.3 Cost control: per-request billing, budget caps, provider cost optimization, caching

9.3.4 Auditability: full request/response logging, screenshot evidence, compliance reporting

10. Phase 9: Build & Automation Integration (~2000 words, 2 tables)

10.1 Makefile Targets

10.1.1 make qa-all: run unit → integration → e2e → screenshot-verify → anti-bluff → challenges → report

10.1.2 make qa-session: run full autonomous QA session with all platforms

10.1.3 make qa-anti-bluff: run deliberate-break validation suite

10.1.4 make qa-screenshot: capture screenshots for all client apps on demand

10.1.5 make qa-report: generate consolidated QA report from latest session results

10.2 Session Scripts

10.2.1 scripts/run_full_qa.sh: orchestrates complete QA cycle with resource limits and timeout

10.2.2 scripts/run_nightly_qa.sh: scheduled heavy QA session with full platform coverage

10.2.3 scripts/verify_screenshots.sh: validates all screenshots are non-empty, non-uniform, contain expected UI elements

10.2.4 scripts/run_all_challenges.sh: extended to include anti-bluff and screenshot challenges

10.3 No-CI/CD Compliance

10.3.1 All automation via Makefile and shell scripts per constitutional mandate (NO .github/workflows/)

10.3.2 Manual trigger model: operator runs make qa-all after significant changes

10.3.3 Pre-commit hooks prohibited; use local verification via scripts/pre-validate.sh instead

10.3.4 Build orchestrator (make build → ./bin/) owns container lifecycle; no manual docker commands

11. Phase 10: Monitoring, Reporting & Compliance (~2000 words, 2 tables)

11.1 QA Results Dashboard

11.1.1 Static HTML dashboard: challenges-dashboard over qa-results/ with trend charts, platform coverage, issue severity

11.1.2 Ticket viewer: static HTML renderer for OCU tickets with inline evidence screenshots

11.1.3 Coverage tracker: per-feature, per-platform, per-test-type coverage percentages with historical trends

11.2 Compliance Reporting

11.2.1 Anti-bluff compliance report: per-test verification method, deliberate-break test results, false-positive rate

11.2.2 Governance compliance report: submodule constitution verification, cascade status, missing mandates

11.2.3 Release gate report: all tests pass + all challenges pass + all screenshots verified + all anti-bluff checks pass

11.3 Continuous Improvement

11.3.1 Feedback loop: failed tests → ticket generation → fix → new Challenge → regression guard

11.3.2 Test bank expansion: add new test cases for every bug found, every feature added, every provider integrated

11.3.3 Benchmark baselines: track test execution time, screenshot capture latency, LLM response time per provider

11.3.4 Monthly review: analyze QA trends, coverage gaps, most fragile components, highest-impact improvements

References

Research Artifacts

- **Type:** Deep research outputs
- **Description:** Six parallel research analyses of HelixCode, HelixQA, Catalogizer, governance, UX, and screenshot/testing strategy
- **Path:** /mnt/agents/output/research/

Source Repositories

- **HelixCode:** <https://github.com/HelixDevelopment/HelixCode>
- **HelixQA:** <https://github.com/HelixDevelopment/HelixQA>
- **Catalogizer:** <https://github.com/vasic-digital/Catalogizer>